Theoretical Guide
py.Sandu

Daniel Oliveira, Gustavo Alves & Lucas Mendes

1 Number Theory

1.1 Modular Arithmetic & Number Theory

- **Properties of Modulo:**
  - Sum: $(a+b)\%m = (a\%m + b\%m)\%m$
  - Subtraction: $(a-b)\%m = (a\%m - b\%m + m)\%m$
  - Product: $(a \times b)\%m = (a\%m \times b\%m)\%m$
- **Modular Division:** $(a/b) \equiv (a \times b^{-1}) \pmod{m}$, where $b \times b^{-1} \equiv 1 \pmod{m}$. Valid if $\gcd(b,m)=1$.
- **Euler's Totient Theorem:** If $\gcd(a,m)=1$, then $a^{\phi(m)} \equiv 1 \pmod{m}$.
- **Fermat's Little Theorem:** If p is prime, $a^{p-1} \equiv 1 \pmod{p}$. Inverse: $a^{-1} \equiv a^{p-2} \pmod{p}$.
- **Power Reduction:** If p is prime: $a^b \equiv a^{b \pmod{p-1}} \pmod{p}$. General: $a^b \equiv a^{b \pmod{\phi(m)}} \pmod{m}$ (if $\gcd(a,m)=1$).
- **Bézout's Identity:** For a, b, there exist x, y such that $ax + by = \gcd(a,b)$.

1.2 Divisors and Prime Factorization

For an integer $n > 1$, let its prime factorization be $n = p_1^{a_1} \cdot p_2^{a_2} \dots p_k^{a_k}$.

- **Number of Divisors (τ):** The total count of divisors of n.

$$\tau(n) = \prod_{i=1}^{k} (a_i + 1)$$

- **Sum of Divisors (σ):** The sum of all positive divisors of n.

$$\sigma(n) = \prod_{i=1}^{k} \left(\frac{p_i^{a_i+1} - 1}{p_i - 1} \right)$$

- **Product of Divisors (P):** The product of all divisors of n.

$$P(n) = n^{\frac{\tau(n)}{2}}$$

Note: If n is a perfect square, $\tau(n)$ is odd, but the result remains an integer.

- **Euler's Totient Function (ϕ):** Number of integers $k \in [1,n]$ such that $\gcd(k,n) = 1$.

$$\phi(n) = n \prod_{i=1}^{k} \left(1 - \frac{1}{p_i} \right)$$

2 Geometry

2.1 Lines & Polygons

- **Slope-Intercept:** $y = mx + c$
- **General Line Equation:** $Ax + By + C = 0$
- **Distance from point (x_0, y_0) to line $Ax + By + C = 0$:** $d = \frac{|Ax_0 + By_0 + C|}{\sqrt{A^2+B^2}}$
- **Shoelace Formula (Area of Polygon):** $A = \frac{1}{2} | \sum_{i=1}^{n-1} (x_i y_{i+1} - x_{i+1} y_i) + (x_n y_1 - x_1 y_n)|$
  OBS: All vertices must be in order, either clockwise or counterclockwise
- **Pick's Theorem (Area of Lattice Polygons):** $A = I + \frac{B}{2} - 1$, where I is interior points and B is boundary points

2.2 Circles

2.3 Circles

- **Area:** πr^2
- **Circumference:** $2\pi r$
- **Equation:** $(x-h)^2 + (y-k)^2 = r^2$

2.4 Spheres

- **Volume:** $\frac{4}{3}\pi r^3$
- **Surface Area:** $4\pi r^2$
- **Equation:** $(x-h)^2 + (y-k)^2 + (z-l)^2 = r^2$

2.5 Basic 3D Geometry

- **Distance Formula:** $\sqrt{\Delta x^2 + \Delta y^2 + \Delta z^2}$
- **Vector Definition:** $\vec{v} = (x, y, z)$
- **Dot Product** $(\vec{a} \cdot \vec{b})$**:** $x_1x_2 + y_1y_2 + z_1z_2 = |\vec{a}||\vec{b}|\cos\theta$
  Usage: Finding angles, checking perpendicularity $(\vec{a} \cdot \vec{b} = 0)$
- **Cross Product** $(\vec{a} \times \vec{b})$ $:(y_1z_2 - z_1y_2, z_1x_2 - x_1z_2, x_1y_2 - y_1x_2)$
  Usage: Results in a 3D vector perpendicular to both $\vec{a}$ and $\vec{b}$. Magnitude is the area of the parallelogram formed by $\vec{a}$ and $\vec{b}$.
- **Triple Scalar Product** $(\vec{a} \cdot (\vec{b} \times \vec{c}))$**:** Determinant of the 3×3 matrix of the vectors.
  Usage: Volume of a parallelepiped, checking coplanarity (Volume $= 0$).
- **Volumes:**
  - **Sphere:** $\frac{4}{3}\pi r^3$
  - **Cylinder:** $\pi r^2 h$
  - **Cone:** $\frac{1}{3}\pi r^2 h$
  - **Tetrahedron (from 3 vectors):** $\frac{1}{6}|\vec{a} \cdot (\vec{b} \times \vec{c})|$

2.6 Basic 2D Geometry

- **Distance Formula:** $\sqrt{\Delta x^2 + \Delta y^2}$
- **Vector Definition:** $\vec{v} = (x, y)$
- **Dot Product** $(\vec{a} \cdot \vec{b})$**:** $x_1x_2 + y_1y_2 = |\vec{a}||\vec{b}|\cos\theta$
  Usage: Finding angles, checking perpendicularity $(\vec{a} \cdot \vec{b} = 0)$
- **Cross Product** $(\vec{a} \times \vec{b})$ $:x_1y_2 - y_1x_2 = |\vec{a}||\vec{b}|\sin\theta$
  Usage: 2D orientatiom, calculating signed area, collinearity $(\vec{a} \times \vec{b} = 0)$
- **Midpoint:** $M = (\frac{x_1+x_2}{2}, \frac{y_1+y_2}{2})$

2.7 Algorithms/Concepts

- **Convex Hull (Monotone Chain or Graham Scan):** Finds the smallest convex polygon containing all points
- **Line Segment Intersection:** Using cross products to check if two segments intersect
- **Point in Polygon:** Using ray casting (winding number) algorithm for O(n) complexity or BSP Tree for O(log n) complexity
- **Angle of vector:** Use `atan2(y,x)` to find the angle of a vector.

3 Progressions

- **Arithmetic Progression (AP):** Sum of the first n terms:

$$\sum_{i=1}^{n} i = 1 + 2 + \cdots + n = \frac{n(n+1)}{2}$$

- **Sum of Squares:** Sum of the first n squares:

$$\sum_{i=1}^{n} i^2 = 1^2 + 2^2 + \cdots + n^2 = \frac{n(n+1)(2n+1)}{6}$$

- **Sum of Cubes:** Sum of the first n cubes:

$$\sum_{i=1}^{n} i^3 = 1^3 + 2^3 + \cdots + n^3 = \left(\frac{n(n+1)}{2}\right)^2$$

- **Geometric Progression (GP):** Sum of the first n terms (ratio $r \neq 1$):

$$\sum_{i=0}^{n-1} a \cdot r^i = a\left(\frac{r^n - 1}{r - 1}\right)$$

  Note: If $|r| < 1$, the infinite sum is $S_\infty = \frac{a}{1-r}$.

- **Arithmetico-Geometric Series:** Sum of $\sum_{i=1}^{n} i \cdot r^{i-1}$ (common in expected value problems):

$$\sum_{i=1}^{n} i \cdot r^{i-1} = \frac{1-r^n}{(1-r)^2} - \frac{nr^n}{1-r}$$

4 Bitwise

4.1 Techniques

- **Check Even/Odd**: $n\&1$ can be used to check if a number is even (result is 0) or odd (result is 1).
- **Power of 2 check:** $(n\&\&!(n\&(n-1)))$ returns true if n is a power of 2.
- **Count 1s:** To count the number of 1s in the binary representation of a number, you can use the expression $n\&(n-1)$ repeatedly until n becomes 0. Each time you perform this operation, it removes the rightmost 1 from the binary representation, allowing you to count how many times you can do this before n becomes 0.
- **Isolate/Clear Rightmost Set Bit:** To isolate the rightmost set bit of a number, you can use the expression $n\&(-n)$. To clear the rightmost set bit, you can use $n\&(n-1)$.
- **Subset Generation:** Use a loop from 0 to $2^n - 1$ to generate all subsets of a set of size n. Each number in this range represents a subset, where the binary representation indicates which elements are included 1 or excluded 0.

4.2 Algebraic Properties

- **Addition Relationship:** $a + b = (a \oplus b) + 2 \cdot (a \wedge b)$
- **XOR Relationship:** $a|b = (a \oplus b) + (a \wedge b)$
- **AND Relationship:** $a \wedge b = (a + b) - (a \oplus b)$
- **Range XOR:** The XOR of all numbers from 1 to n follows a pattern based on $n \pmod 4$:
  - If $n \mod 4 = 0$, then XOR sum $= n$
  - If $n \mod 4 = 1$, then XOR sum $= 1$
  - If $n \mod 4 = 2$, then XOR sum $= n + 1$
  - If $n \mod 4 = 3$, then XOR sum $= 0$

5 Math

5.1 Graphs

General Properties

- **Handshaking Lemma:** The sum of the degrees of all vertices is exactly twice the number of edges ($\sum \deg(v) = 2|E|$).
- **Bipartite Graph:** A graph whose vertices can be divided into two disjoint sets such that every edge connects a vertex in one set to a vertex in the other. A graph is bipartite if and only if it is **2-colorable** and contains **no cycles of odd length**.
- **Complete Graph (K_n):** A simple graph where every pair of vertices is connected by an edge. It contains exactly $\frac{N(N-1)}{2}$ edges.
- **DAG (Directed Acyclic Graph):** A directed graph with no directed cycles. Every DAG has at least one valid *Topological Ordering*.
- **Planar Graph:** A graph that can be drawn without edges crossing. **Euler's Formula:** For a planar graph with V vertices, E edges, F faces, and C connected components: $V - E + F = 1 + C$.

Tree Properties

- **Definition & Edges:** A tree is a connected graph with no cycles. A tree with N vertices always has exactly $N - 1$ edges.
- **Unique Paths:** There is exactly one simple path between any two vertices in a tree.
- **Cycle Creation:** Adding exactly one new edge to any tree will create exactly one cycle.
- **Bipartite Nature:** Every tree is inherently a bipartite graph (since it has no cycles, it has no odd cycles).

Important Graph Theorems

- **Eulerian Cycle & Path:**

- An undirected connected graph has an **Eulerian Cycle** (visits every edge exactly once and returns to start) $\iff$ **every** vertex has an even degree.
- It has an **Eulerian Path** (visits every edge exactly once but doesn't return) $\iff$ exactly **zero or two** vertices have an odd degree.

- **Dirac's Theorem (Hamiltonian Cycle):** In a simple graph with $N \geq 3$ vertices, if every vertex has a degree $\deg(v) \geq \frac{N}{2}$, the graph is guaranteed to have a Hamiltonian cycle (visits every vertex exactly once).
- **Ore's Theorem (Hamiltonian Cycle):** In a simple graph with $N \geq 3$ vertices, if $\deg(u)+\deg(v) \geq N$ for every pair of non-adjacent vertices u and v, the graph contains a Hamiltonian cycle.
- **Mantel's Theorem:** The maximum number of edges in an N-vertex graph that does not contain any triangles (cycles of length 3) is strictly bounded by $\lfloor \frac{N^2}{4} \rfloor$.
- **Kőnig's Theorem:** In any **bipartite graph**, the size of the Maximum Bipartite Matching is exactly equal to the size of the Minimum Vertex Cover. *(Reduces an NP-Hard problem to a Max Flow / Bipartite Matching problem).*
- **Dilworth's Theorem:** In any DAG, the size of the maximum anti-chain (largest set of vertices where none can reach another) is equal to the minimum number of directed paths needed to cover all vertices.

6 Constants

6.1 Complexity Estimates & Tactical Limits

- **1 second limit:** 1 second is usually enough for a program to perform around 10^8 operations.
- **256Mb limit:** 256 megabytes is usually enough to store around 10^7 integers.
- **Common Complexity Thresholds (for 1s time limit):**
  - $N \leq 10$: $O(N!)$ or $O(N^2 \cdot N!)$ (Permutations, brute force).
  - $N \leq 20$: $O(2^N \cdot N^k)$ (Subset DP, Bitmask DP).
  - $N \leq 500$: $O(N^3)$ (Floyd-Warshall, matrix multiplication).
  - $N \leq 5000$: $O(N^2)$ (Simple DP, nested loops).
  - $N \leq 2 \cdot 10^5$: $O(N \log N)$ (Sorting, Segment Tree, DSU, Dijkstra). **Note:** $O(N \log^2 N)$ also usually passes.
  - $N \leq 10^6$: $O(N)$ or $O(N \log N)$ with very small constants.
  - $N > 10^7$: $O(\log N)$ or $O(1)$ (Binary search, math formulas, matrix exponentiation).
- **Specific Bounds for Math:**
  - **Factorials:** 12! fits in 32-bit `int`, 20! fits in 64-bit `long long`.
  - **Powers of 2:** $2^{31} - 1 \approx 2 \cdot 10^9$ (`int`), $2^{63} - 1 \approx 9 \cdot 10^{18}$ (`long long`).
  - **Fibonacci:** F_{46} is the last to fit in 32-bit `int`, F_{92} in 64-bit `long long`.

6.2 Data type limits

- **Integer limits**:
  - `int` (32-bit): $\approx \pm 2.14 \times 10^9$.
  - `long long` (64-bit): $\approx \pm 9.22 \times 10^{18}$.
  - `__int128_t`: $\approx \pm 1.7 \times 10^{38}$
- **Floating-point precision**:
  - `float`: 7 decimal digits.
  - `double`: 15 decimal digits
  - `long double`: 18-19 decimal digits

7 Counting Problems

7.1 Permutation

- **Definition:** A permutation of a set is an arrangement of its members into a sequence or linear order. The number of permutations of n distinct objects is given by $n!$ (n factorial).
- **Permutations with Repetition:** If there are n objects where there are n_1 indistinguishable objects of one type, n_2 indistinguishable objects of another type, and so on, the number of distinct permutations is given by $\frac{n!}{n_1! \cdot n_2! \cdot ...}$.

- **Circular Permutations:** The number of ways to arrange n distinct objects in a circle is given by $(n-1)!$
- **Circular Permutations with Repetition:** . Let $g = \gcd(n_1, n_2, \dots)$. The total number of arrangements is given by:

$$C = \frac{1}{n} \sum_{d|g} \phi(d) \frac{(n/d)!}{(n_1/d)! \cdot (n_2/d)! \dots}$$

7.2 Combination

- **Definition:** A combination is a selection of items from a larger set, where the order of selection does not matter. The number of combinations of n distinct objects taken r at a time is given by the formula:

$$C(n,r) = \frac{n!}{r!(n-r)!}$$

- **Combinations with Repetition:** If repetition of items is allowed, the number of combinations of n distinct objects taken r at a time is given by the formula:
$$C(n+r-1,r) = \frac{(n+r-1)!}{r!(n-1)!}$$

- **Pascal's Triangle:** The number of combinations can also be found using Pascal's Triangle, where the entry in the n-th row and r-th column corresponds to $C(n,r)$.

8 Identities

9 C++

9.1 System optimization

9.2 Build System (Makefile)

Para compilar qualquer arquivo (ex: `make A` para o arquivo `A.cpp`) com flags de debug, *sanitizers* e a macro `LOCAL` ativa, utilize o seguinte `Makefile`:

```
# Variaveis implicitas do Make
CXX = g++
CXXFLAGS = -Wall -Wconversion -Wfatal-errors -g -std=c++17
 -DLOCAL -fsanitize=undefined,address
LDFLAGS = -fsanitize=undefined,address
```

9.3 String

- `s.find(str)`: Retorna o índice da primeira ocorrência da substring `str`. **Nota sobre npos:** Se a substring não for encontrada, a função retorna a constante `string::npos`. A verificação para saber se a string existe é `if (s.find(str) != string::npos)`.
- `s.substr(pos, len)`: Retorna a substring que começa no índice `pos` e tem tamanho `len`. Se o parâmetro `len` for omitido, a substring vai de `pos` até o final da string original.
- `s.erase(pos, len)`: Remove `len` caracteres a partir do índice `pos`. Altera a string original.
- `s.insert(pos, str)`: Insere o conteúdo de `str` exatamente a partir do índice `pos`.
- `to_string(val)`: Converte números (int, double, long long) para `string`.
- `stoi(s)` / `stoll(s)`: Converte `string` para `int` ou `long long`, respectivamente.
- `isalpha(c)` / `isdigit(c)` / `isalnum(c)`: Retorna verdadeiro se o *char* for uma letra, um dígito, ou alfanumérico.
- `tolower(c)` / `toupper(c)`: Converte o *char* para minúsculo ou maiúsculo. (Para converter a string toda: `for(char &c : s) c = tolower(c);`).
- **Leitura até o EOF (Fim de Arquivo):**

```
string s;
while (cin >> s) {
    // Processa cada palavra isoladamente
}
```

- **Lendo uma linha inteira e separando por espaços (<sstream>):.**

```
#include <sstream>
string linha, palavra;
getline(cin, linha); // Lê a linha toda, incluindo espaços
```

```
    stringstream ss(linha);
    while (ss >> palavra) {
        // 'palavra' vai receber cada token separado por espaço
    }
```

9.4 Built-ins do GCC (Operações Bitwise)

- `__builtin_popcount(x)`: Conta o número de bits '1' no inteiro `x`. Para `long long`, use `__builtin_popcountll(x)`.
- `__builtin_clz(x)`: Conta os zeros à esquerda (*count leading zeros*).
- `__builtin_ctz(x)`: Conta os zeros à direita (*count trailing zeros*).